

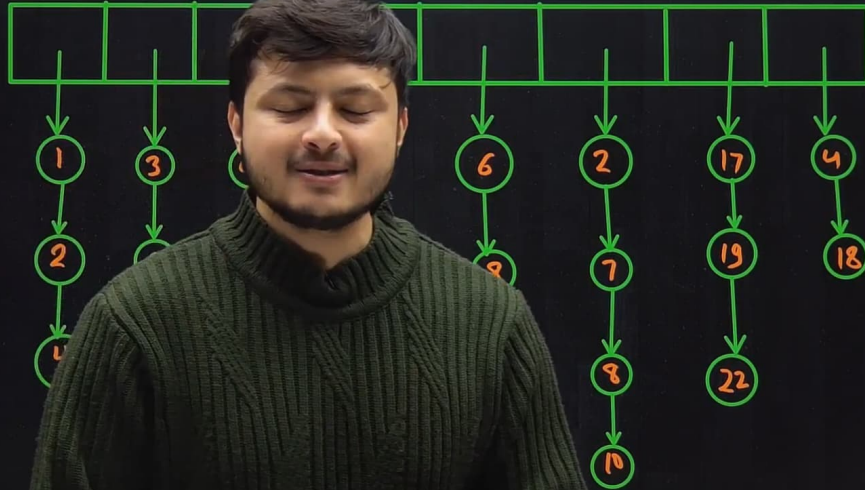
Merge K Sorted Linked List

273 693

16:41

K

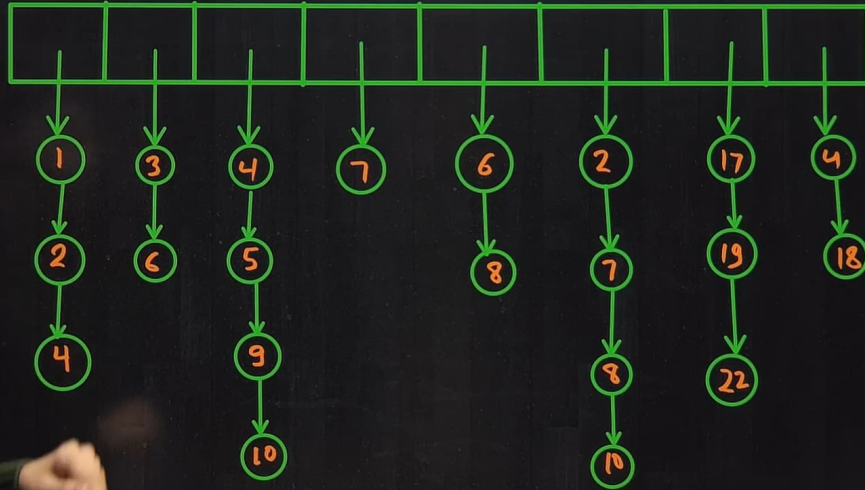
Q88



Merge K Sorted Linked List

273 693
16:41

K



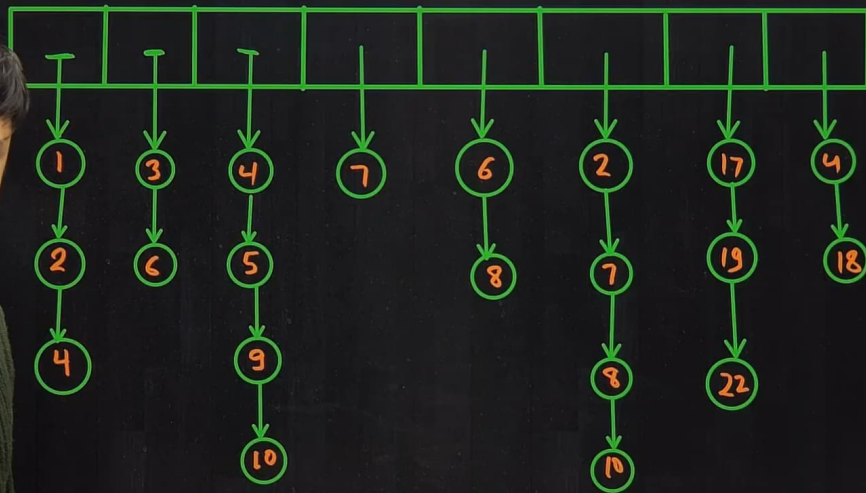
Merge K Sorted Linked List

273 693

16:42

K

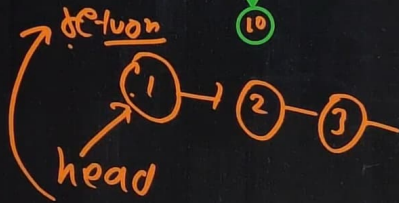
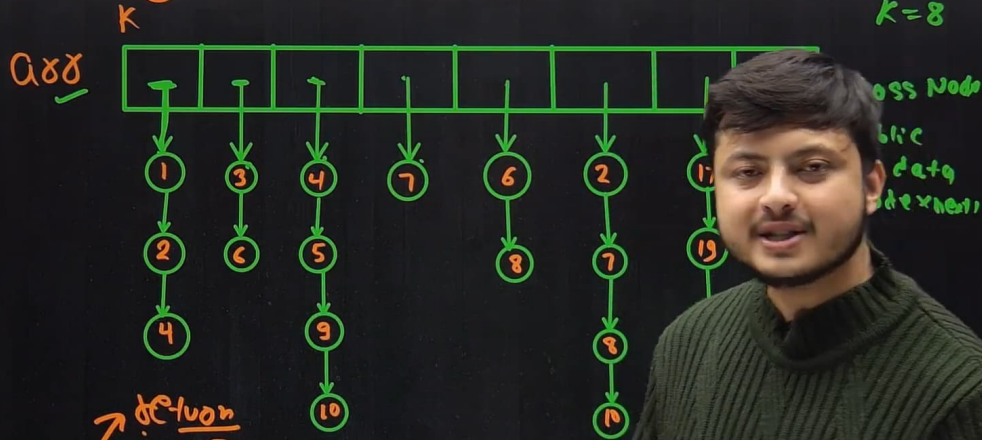
K=8



Merge K Sorted Linked List

535 837

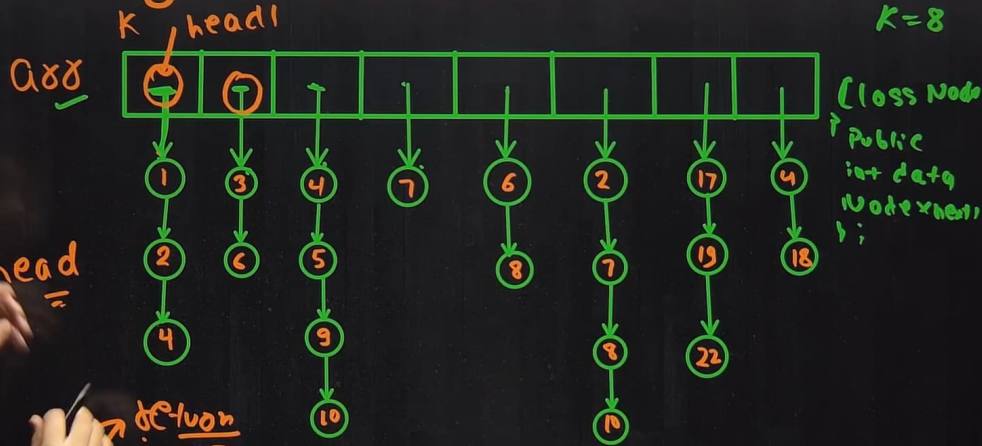
16:44



Merge K Sorted Linked List

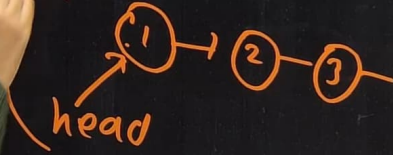
535 837

16:44



head =

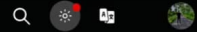
function



```
node * head = arr[0];  
for (i = 1; i < K; i++)  
{  
    head = merge(head, arr[i]);  
}  
  
return head;
```

90% Money-Back!

Courses ▾ Tutorials ▾ Jobs ▾ Practice ▾ Contests ▾



</> Problem

Editorial

Submissions

Comments

C++ (g++ 5.4)

Average Time: 60m

Start Timer



Merge K sorted linked lists



Medium

Accuracy: 57.01%

Submissions: 74K+

Points: 4

30+ People have Claimed their 90% Refunds. Start Your Journey Today!



Given K sorted linked lists of different sizes. The task is to merge them in such a way that after merging they will be a single sorted linked list.

Example 1:

Input:

K = 4

value = {{1,2,3},{4 5},{5 6},{7,8}}

Output: 1 2 3 4 5 5 6 7 8

Explanation:

The test case has 4 sorted linked

list of size 3, 2, 2, 2

1st list 1 -> 2-> 3

2nd list 4->5

3rd list 5->6

4th list 7->8

The merged list will be

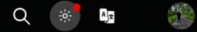
1->2->3->4->5->5->6->7->8.

```
39     data = x;
40     next = NULL;
41 }
42
43 };
44 */
45
46 class Solution{
47 public:
48
49
50     //Function to merge K sorted linked list.
51     Node * mergeKLists(Node *arr[], int K)
52     {
53         // Your code here
54         |
55     }
56 };
57
58
59 // } Driver Code Ends
```

{Three
90
Challenge}

90% Money-Back!

Courses Tutorials Jobs Practice Contests



Problem Editorial Submissions Comments

C++ (g++ 5.4)

Average Time: 60m

Start Timer



Merge K sorted linked lists



Medium Accuracy: 57.01% Submissions: 74K+ Points: 4

30+ People have Claimed their 90% Refunds. Start Your Journey Today!

Given K sorted linked lists of different sizes. The task is to merge them in such a way that after merging they will be a single sorted linked list.

Example 1:

Input:

K = 4

value = {{1,2,3},{4 5},{5 6},{7,8}}

Output: 1 2 3 4 5 5 6 7 8

Explanation:

The test case has 4 sorted linked

list of size 3, 2, 2, 2

1st list 1 -> 2-> 3

2nd list 4->5

3rd list 5->6

4th list 7->8

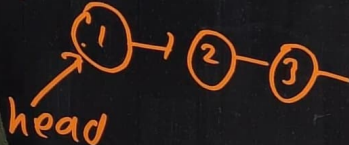
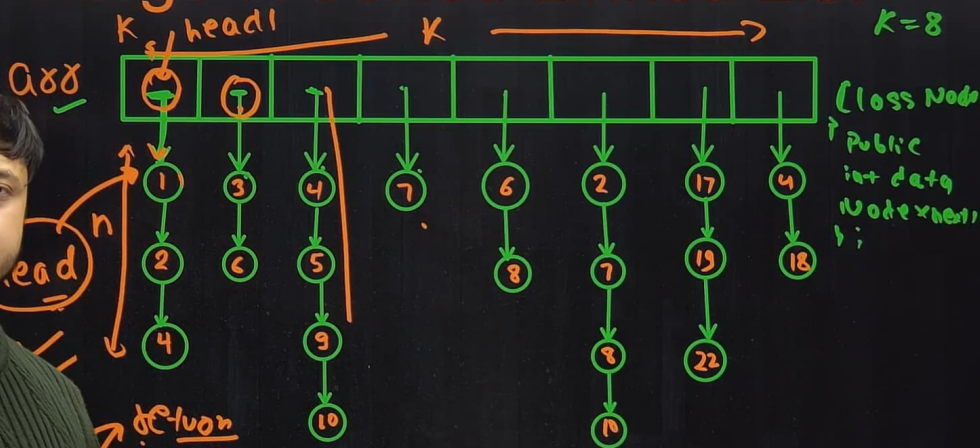
The merged list will be

1->2->3->4->5->5->6->7->8.

```
44 *//
45
46 class Solution{
47     public:
48
49
50     //Function to merge K sorted linked list.
51     Node * mergeKLists(Node *arr[], int K)
52     {
53         // Your code here
54         Node *head = arr[0];
55
56         for(int i=1;i<K;i++)
57         {
58             head = merge(head,arr[i]);
59         };
60
61         return head;
62     }
63 };
64
```

{Three
90
Challenge}

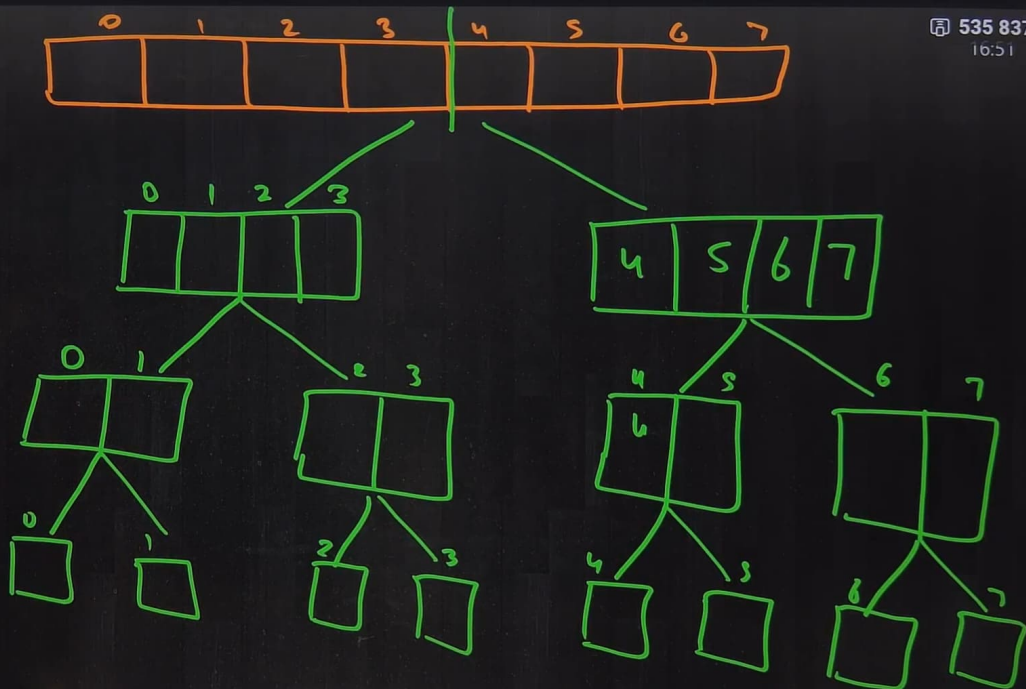
Merge K Sorted Linked List

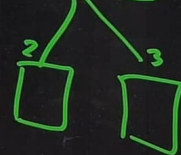
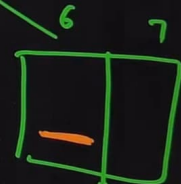
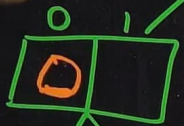
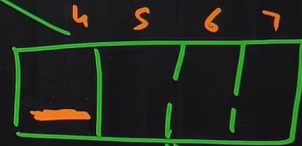
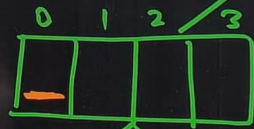
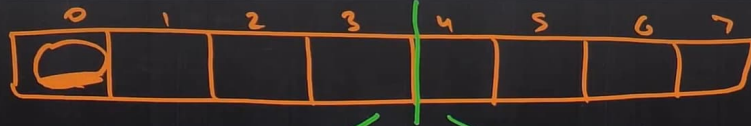


$$2N + 3N + 4N + \dots + KN$$

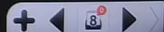
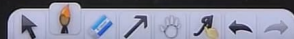
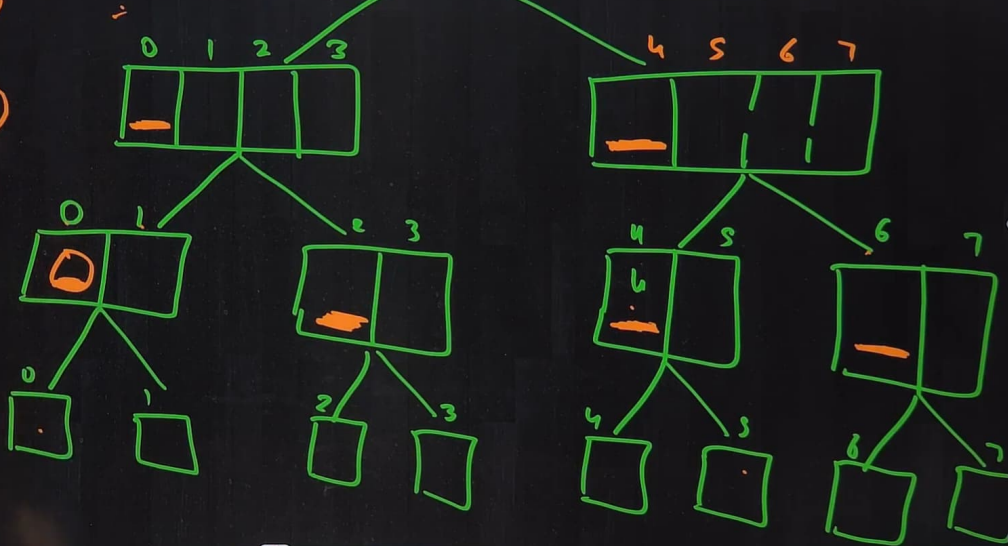
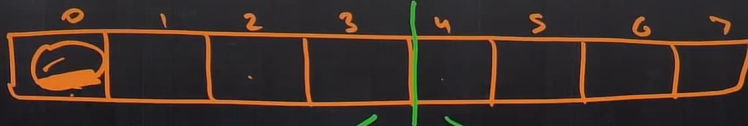
$$N(2 + 3 + 4 + \dots + K)$$

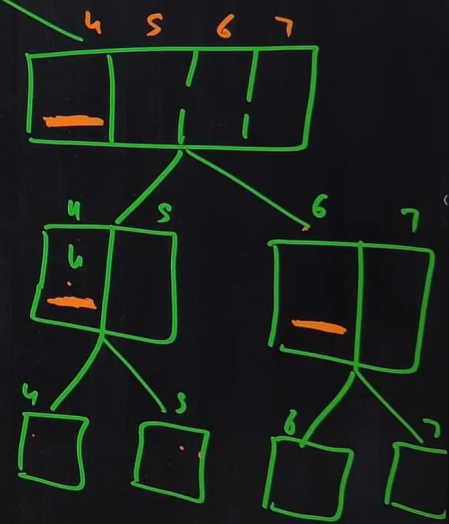
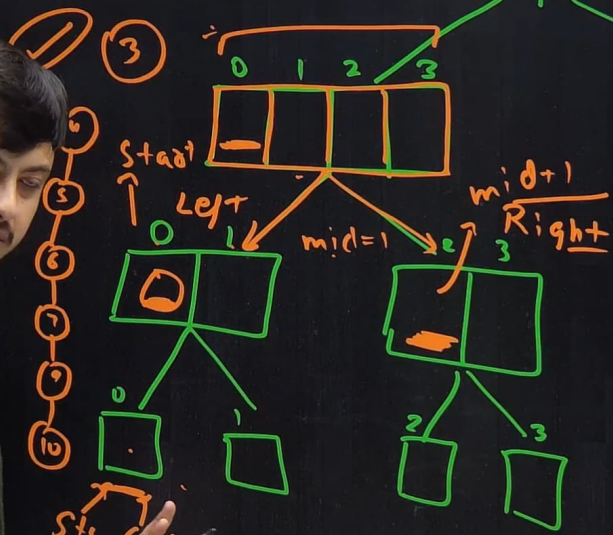
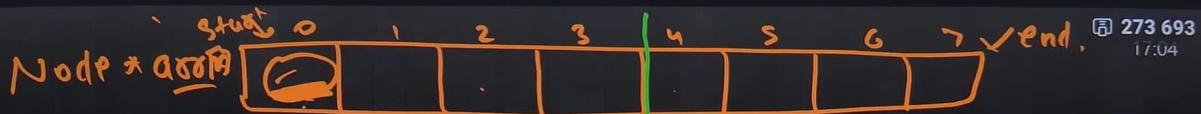
$$O(K^2 N)$$





Node * root



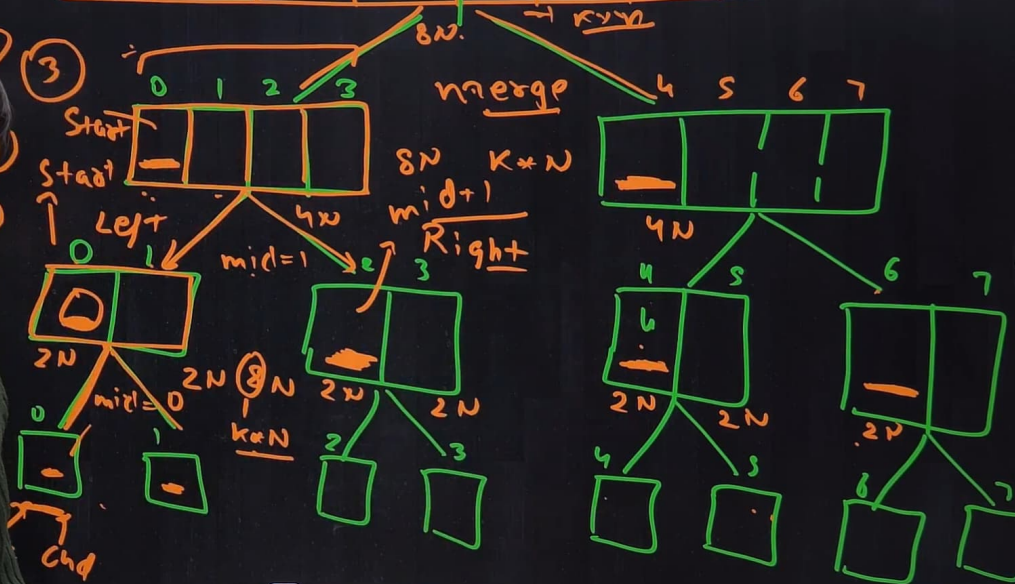


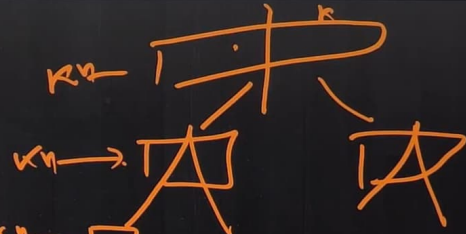
273 693
17:05

```
mergesort(Node *arr[], int start, int end)
{
    if (start >= end)
        return;
```

```
    int mid = start + (end - start) / 2;
    mergesort(arr, start, mid);
    mergesort(arr, mid + 1, end);
    arr[start] = merge(arr[start], arr[mid + 1]);
}
```


Node * ans





$$k \times n + k \times n + k \times n + \dots$$

$$k \times n \times \log k$$

$$\boxed{nk \log k}$$

Merge K sorted linked lists

Medium Accuracy: 57.01% Submissions: 74K+ Points: 4

30+ People have Claimed their 90% Refunds. Start Your Journey Today!

Given K sorted linked lists of different sizes. The task is to merge them in such a way that after merging they will be a single sorted linked list.

Example 1:

Input:

K = 4

value = {{1,2,3},{4 5},{5 6},{7,8}}

Output: 1 2 3 4 5 5 6 7 8

Explanation:

The test case has 4 sorted linked

list of size 3, 2, 2, 2

1st list 1 -> 2-> 3

2nd list 4->5

3rd list 5->6

4th list 7->8

The merged list will be

1->2->3->4->5->5->6->7->8.

C++ (g++ 5.4) Average Time: 60m Start Timer

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
```

```
if(head1)
tail->next = head1;
else
tail->next = head2;

// Dummy node ko pehle delete maaro
return head->next;
}

//Function to merge K sorted linked list.
Node * mergeKLists(Node *arr[], int K)
{
    // Your code here
    mergesort(arr,0,K-1);
    return arr[0];
};
```



Merge K sorted linked lists

Medium Accuracy: 57.01% Submissions: 74K+ Points: 4

30+ People have Claimed their 90% Refunds. Start Your Journey Today!

Given K sorted linked lists of different sizes. The task is to merge them in such a way that after merging they will be a single sorted linked list.

Example 1:

Input:

K = 4

value = {{1,2,3},{4 5},{5 6},{7,8}}

Output: 1 2 3 4 5 5 6 7 8

Explanation:

The test case has 4 sorted linked

list of size 3, 2, 2, 2

1st list 1 -> 2-> 3

2nd list 4->5

3rd list 5->6

4th list 7->8

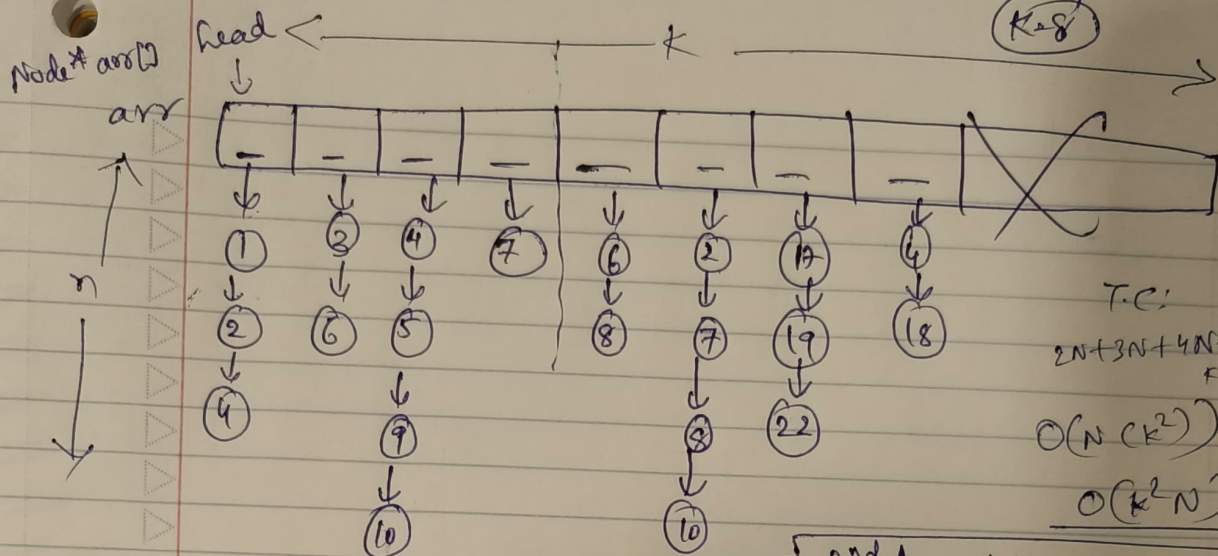
The merged list will be

1->2->3->4->5->5->6->7->8.

```
81 }
82 void mergesort(Node *arr[], int start, int end)
83 {
84     if(start>=end)
85         return;
86
87     int mid = start+(end-start)/2;
88     // left part
89     mergesort(arr,start,mid);
90     // Right part
91     mergesort(arr,mid+1,end);
92     arr[start]=merge(arr[start],arr[mid+1]);
93 }
94
95 //Function to merge K sorted linked list.
96 Node * mergeKLists(Node *arr[], int K)
97 {
98     // Your code here
99     mergesort(arr,0,K-1);
100     return arr[0];
101 }
```

{Three
90
Challenge}

Merge K Sorted LL



Using similar/same
Approach we followed
to solve previous problem:

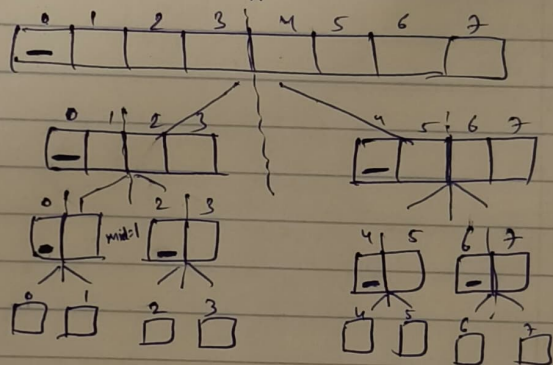
```
Node* head = arr[0]
for (int i = 1; i < K; i++) {
```

```
    head = merge(head, arr[i]);
```

```
}
return head;
```

Same
as previous
only instead of
bottom,
we use *next

2nd Approach:



```
mergeSort(Node* arr[], int start, int end) {
```

```
    if (start >= end)
        return
```

```
    int mid = start + (end - start) / 2;
```

```
    mergeSort(arr, start, mid);
```

```
    mergeSort(arr, mid+1, end);
```

```
    arr[start] = merge(arr[start], arr[mid+1]);
```

```
}
```